

Spike: Task 16

Title: Sound Board

Author: Mohamed Shafi Uzman Fassy, 102608927

Goals / deliverables:

The goal of this spike task is to demonstrate my ability to use a framework to load, play and control game sound and music using SDL2 and SDL2 mixer.

Besides this report, what else was created?

- Code files in see C:\Users\storm\Documents\GitHub\GAMES PROGRAMMING REPO\COS30031-2023-102608927\16 - Spike - Sound Board\SDLMixerTask\SDLMixerTask.

Technologies, Tools, and Resources used:

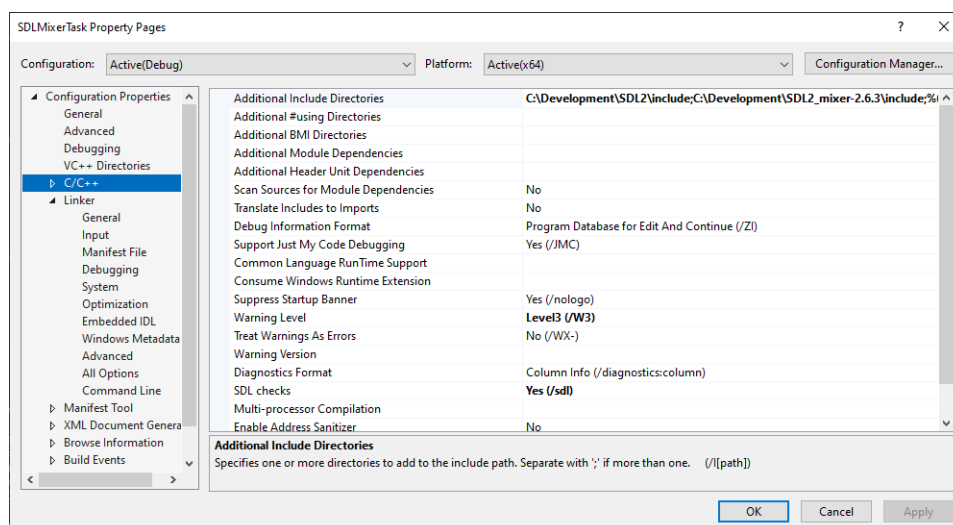
List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module

Tasks undertaken:

- Watching SDL2 Mixer Installation Guide Videos
- Setting up Project environment to use SDL2 and SDL2 Mixer
- Download 4 sound files.

Implementing SDL2 and SDL2 Mixer:



Once I downloaded the SDL2 and SDL2 mixer libraries I had to add them into the additional include directories in my project properties settings.

How we Initialize the sounds:

```
1  if (SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO) != 0) {  
2      std::cerr << "Error initializing SDL: " << SDL_GetError() << std::endl;  
3      return 1;  
4  }
```

The first step towards playing sounds in SDL is initializing the SDL library itself, with a focus on the video and audio subsystems. This is accomplished with the **SDL_Init()** function.

```
1  if (Mix_OpenAudio(44100, MIX_DEFAULT_FORMAT, 2, 2048) < 0) {  
2      std::cerr << "Error initializing SDL_mixer: " << Mix_GetError() << std::endl;  
3      SDL_DestroyRenderer(renderer);  
4      SDL_DestroyWindow(window);  
5      SDL_Quit();  
6      return 1;  
7  }
```

Once SDL is initialized, we need to set up the SDL_mixer, which is an SDL library extension that allows us to play sound files. To do this, we use the **Mix_OpenAudio()** function. This function sets the audio format for our application. We are initializing the audio rate to 44100Hz, using the default format, setting the output to stereo (2 channels), and using a chunk size of 2048 bytes.

```
1  Mix_Chunk* sound1 = Mix_LoadWAV("1.wav");  
2  Mix_Chunk* sound2 = Mix_LoadWAV("2.wav");  
3  Mix_Chunk* sound3 = Mix_LoadWAV("3.wav");  
4  Mix_Music* bgMusic = Mix_LoadMUS("4.mp3");
```

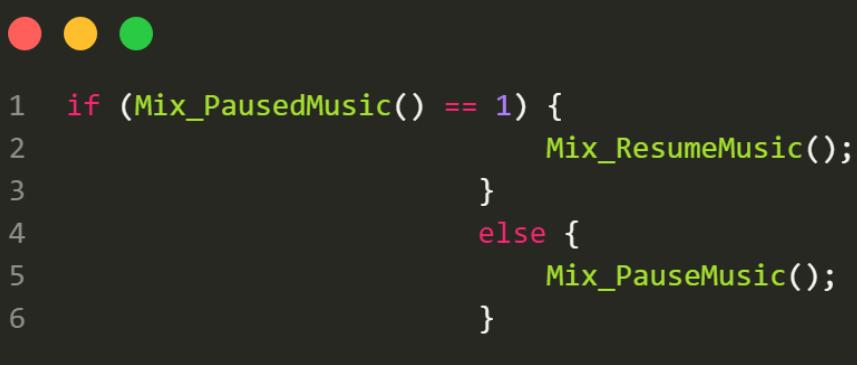
Before playing sounds, we must load them onto the memory. With SDL_mixer, we use the **Mix_LoadWAV()** function for sound effects (short audio clips) and **Mix_LoadMUS()** for music (longer audio tracks).

How we play the sounds:

```
if (event.type == SDL_KEYDOWN) {
    switch (event.key.keysym.sym) {
        case SDLK_1:
            Mix_PlayChannel(-1, sound1, 0);
            break;
        case SDLK_2:
            Mix_PlayChannel(-1, sound2, 0);
            break;
        case SDLK_3:
            Mix_PlayChannel(-1, sound3, 0);
            break;
        case SDLK_0:
            if (Mix_PlayingMusic() == 0) {
                Mix_PlayMusic(bgMusic, -1); // Plays music indefinitely
            }
    }
}
```

Once the sounds are loaded into the memory as chunks, they can be played using the **Mix_PlayChannel()** function. The first argument specifies the channel to play on (-1 means to play on the first free channel), the second argument is the sound chunk to play, and the third argument is the number of times to loop (0 means play once). The music file is played using the **Mix_PlayMusic()** function. The first argument is the music data loaded into memory, and the second argument is the number of times to loop (-1 means loop indefinitely).

How we pause the sounds:



```
1  if (Mix_PausedMusic() == 1) {
2      Mix_ResumeMusic();
3  }
4  else {
5      Mix_PauseMusic();
6  }
```

If there is music playing, the code then checks if the music is currently paused using the **Mix_PausedMusic()** function. Depending on whether the music is paused or not, it either resumes the music using **Mix_ResumeMusic()** or pauses the music using **Mix_PauseMusic()**.